FAST NUCES – Islamabad Campus

Department of Computer Science

A Parallel and Distributed Computing Framework for Real-Time Retail Product Entity Linking and POS data normalisation

Mid-Submission Report

Submitted By

Muhammad Haseeb Arshad (23I-2578)

Muhammad Abdullah Nadeem (23I-2522)

Courses: Parallel and Distributed Computing | Natural Language Processing

Spring 2026

Abstract

Raw point-of-sale (POS) data is consistently undermined by abbreviations, phonetic shorthand, typographical errors, and inconsistent formatting, all of which degrade the reliability of downstream retail analytics. This report presents a five-stage Parallel and Distributed Computing (PDC) framework that integrates specialised Natural Language Processing (NLP) algorithms to resolve noisy POS text strings to canonical Stock Keeping Unit (SKU) identifiers in real time. The framework employs Dask for distributed data preprocessing and Ray Core actors for scalable, stateful inference orchestration. Hallucination is eliminated through outline-guided constrained decoding, which converts the valid product catalogue into a finite-state machine (FSM) that applies bitmask constraints to the LLM's logits at each generation step, guaranteeing every output is a valid catalogue entry. Candidate retrieval uses a hybrid BM25 and bi-encoder search strategy merged via Reciprocal Rank Fusion (RRF), with final reranking performed by a LightGBM classifier over interpretable lexical features. Basket-level context encoding, inspired by the RoCEL row-column paradigm, further improves linking accuracy by exploiting co-purchase signals within individual transactions. Evaluation noise is generated through the nlpaug library, which applies controlled keyboard-distance and character-deletion corruptions to simulate real POS terminal messiness reproducibly.

Table of Contents

Abstract

1. Introduction
    - 1.1 Background and Context
    - 1.2 The Problem: Speed and Accuracy Bottlenecks
    - 1.3 Proposed Solution: A PDC–NLP Pipeline
    - 1.4 Research Objectives
    - 1.5 Scope and Organization of This Report
2. Literature Review and Related Work
    - 2.1 Constraint-Based Decoding for Entity Linking: LLM4BioEL
    - 2.2 Few-Shot Entity Linking Without Fine-Tuning: OneNet
    - 2.3 Table-Structured Entity Linking: RoCEL
    - 2.4 Distributed Computing Infrastructure
    - 2.5 Summary Comparison of Baseline Works
3. Proposed Methodology
    - 3.1 Formal Problem Definition
    - 3.2 System Architecture Overview
    - 3.3 Stage 1: Distributed Ingestion and Stream Partitioning
    - 3.4 Stage 2: Parallel Text Normalization and Context Construction
    - 3.5 Stage 3: Hybrid Retrieval and Feature-Based Reranking
    - 3.6 Stage 4: Outlines-Guided Constrained Generation
    - 3.7 Stage 5: Orchestration with Ray Core and FastAPI
    - 3.8 Evaluation Plan

References

Chapter 1: Introduction

1.1 Background and Context

Modern retail operations depend critically on the integrity and speed of data generated at the point of sale. Every time a customer completes a transaction, a point-of-sale (POS) system records a set of line items describing the products purchased, their quantities, and their prices. Across a single chain of stores, these transactions may number in the hundreds of thousands per day. The aggregate data feeds directly into downstream business intelligence systems: inventory replenishment algorithms, demand forecasting models, supply chain logistics, and customer behaviour analytics, all of which depend upon the POS stream as their primary source of ground truth.

Despite its strategic importance, raw POS data is notoriously problematic. Checkout operators work under time pressure and rely on freeform text entry, producing a corpus riddled with cryptic abbreviations, phonetic shorthand, and inconsistent formatting. A single product such as Coca-Cola 12-ounce may appear as "CK 12oz", "cke 12", "coca 12oz", or "CK12OZ" depending on the operator and terminal. Typographical errors further compound the issue, meaning naive string-matching systems silently inflate the apparent product catalogue while undermining the accuracy of every report that relies on product identity.

The process of resolving these noisy strings to canonical product identifiers – a problem known in the NLP literature as 'entity linking' (EL) – is a foundational requirement for reliable retail intelligence. Entity linking serves as a bridge between the raw, unstructured POS note and the structured product knowledge base, which is typically keyed by Stock Keeping Unit (SKU) identifiers.

1.2 The Problem: Speed and Accuracy Bottlenecks

Recent advances in large language models (LLMs) make them natural candidates for POS normalisation. However, deploying LLMs in a production retail environment exposes two critical failure modes.

The first is latency. Serial LLM inference does not scale linearly with demand. Hundreds of simultaneous peak-hour transactions must each be normalised and linked before inventory databases update a requirement that necessitates parallelisation.

The second is hallucination. LLMs generate text without any structural guarantee that the output corresponds to a real catalogue entry. Phantom SKUs propagate silently into downstream systems, corrupting inventory counts and financial records – a correctness problem, not merely a performance one.

1.3 Proposed Solution: A PDC–NLP Pipeline

This project proposes a five-stage PDC framework integrating accessible, undergraduate-implementable NLP algorithms to overcome both failure modes while maintaining technical rigour.

To address latency, the framework uses Dask [4, 8] for distributed preprocessing and Ray Core [5] actors as a simpler, container-free alternative to Ray Serve or KubeRay. Ray Core provides three primitives – Tasks (stateless functions), Actors (stateful classes), and Objects (shared-memory references) – that are sufficient to build a low-latency, multi-core inference pipeline exposed via a lightweight FastAPI server.

Retrieval combines BM25 (lexical matching) with a bi-encoder (semantic similarity), merging results using Reciprocal Rank Fusion (RRF) [13] or a weighted score formula. Reranking uses a LightGBM classifier over interpretable features such as Levenshtein distance and Jaccard similarity, providing a plug-and-play alternative to computationally expensive cross-attention encoders.

To address hallucination, the framework adopts outline-guided constrained decoding [11]. Outlines converts the valid product catalogue into a Finite-State Machine (FSM) and applies a bitmask to the LLM's logits at each decoding step, ensuring the model can only sample tokens that correspond to valid catalogue entries, reaching a correct-format rate of nearly 100%.

Evaluation noise is generated using the nlpaug library [12], which applies systematic keyboard-distance and character-deletion corruptions to simulate POS terminal messiness without requiring an LLM, giving precise control over the aug_char_p noise parameter. Basket-level context encoding, adapted from RoCEL [3], further improves linking accuracy by exploiting co-purchase signals.

1.4 Research Objectives

- Parallel Preprocessing at Scale: Deploy a Dask-based distributed pipeline capable of cleaning and normalising one million POS notes in under ten minutes.
- Zero Hallucination Guarantee: Implement outline-guided FSM-constrained decoding such that 100% of model outputs correspond to valid catalogue entries.
- Accurate Candidate Retrieval: Apply hybrid BM25 + bi-encoder retrieval merged via RRF [13], with LightGBM reranking over Levenshtein and Jaccard features.
- Contextual Accuracy Improvement: Apply row-column basket context encoding, targeting a measurable improvement in accuracy @ 1 over a context-free baseline.
- Real-Time Throughput: Achieve end-to-end latency under 500 ms per transaction under realistic concurrent load using Ray Core actors.

1.5 Scope and Organization of This Report

This mid-submission report covers all five stages of the proposed pipeline. Chapter 2 surveys the three primary baseline papers and the distributed computing infrastructure, with a comparative summary table in Section 2.5. Chapter 3 presents the full methodology: a formal problem definition (Section 3.1), the five pipeline stages (Sections 3.3–3.7), and the evaluation plan (Section 3.8).

Chapter 2: Literature Review and Related Work

This chapter surveys the literature forming the foundation of the proposed framework. Sections 2.1–2.3 each summarise a primary baseline paper and discuss its adaptation. Section 2.4 reviews the distributed computing infrastructure. Section 2.5 provides a comparative summary.

2.1 Constraint-Based Decoding for Entity Linking: LLM4BioEL

2.1.1 Overview of the Paper

Lin et al. (2025) [1] introduce LLM4BioEL, a framework adapting general-purpose LLMs for biomedical entity linking. Its primary contribution is restrictive decoding: at each decoding step, a trie over all valid entity names masks out tokens not forming valid prefixes before the softmax is applied. The framework also incorporates an entropy-based retrieval mechanism that augments the prompt with external knowledge when model confidence is low.

2.1.2 Limitations and Gaps

LLM4BioEL targets static medical ontologies and does not address the dynamic catalogue updates inherent in retail nor the concurrency risks introduced by live trie modifications. It also does not consider distributed execution.

2.1.3 Adaptation for This Project

This project replaces the static trie and complex lock-free concurrency mechanisms with a simpler, Python-native read-copy-update (RCU) strategy. Python's Global Interpreter Lock (GIL) guarantees that reassigning a variable reference is an atomic operation. When the product catalogue changes, a worker constructs a fresh, immutable catalogue object in the background and then atomically swaps the reference held by the Ray actor. Ray's distributed reference counting then handles reclamation automatically: the old catalogue version is freed only once all in-flight worker tasks that hold a reference to it have completed. This approach eliminates the need for custom lock-free tries or CAS primitives while providing the same zero-blocking guarantee.

Rather than a custom logit processor, constrained decoding is implemented using the Outlines library [11] (Section 3.6), which converts the candidate shortlist into a finite-state machine providing the structural validity guarantee equivalent to trie-based restrictive decoding.

2.2 Few-Shot Entity Linking Without Fine-Tuning: OneNet

2.2.1 Overview of the Paper

Liu et al. (2024) [2] present OneNet, a framework for entity linking without task-specific fine-tuning. Its key contribution is the Entity Reduction Processor: a bi-encoder [6] retrieves the top-k candidates, which a cross-encoder then re-ranks into a shortlist. A Consensus Judger selects the entity supported by the majority across multiple reasoning chains.

2.2.2 Limitations and Gaps

OneNet's serial bi-encoder retrieval limits throughput under concurrent load. Deep cross-encoders introduce an O(N) latency bottleneck incompatible with sub-500ms retail requirements. Neither issue is addressed in the original work, which targets offline batch evaluation.

2.2.3 Adaptation for This Project

This project replaces OneNet's single-modality retrieval with a hybrid BM25 and bi-encoder strategy. BM25 provides keyword precision for exact tokens such as unit sizes and brand codes (e.g., "12oz"), while the bi-encoder provides semantic flexibility for abbreviations and paraphrases. Scores from both systems are merged using Reciprocal Rank Fusion (RRF) [13] according to the weighted formula:

$$\underline{final_score = \alpha \cdot BM25_score + (1 - \alpha) \cdot Vector_score}$$

where α is tuned on a held-out validation set. The deep cross-encoder reranker is replaced by a LightGBM classifier trained over interpretable lexical features, including Levenshtein edit distance, Jaccard token overlap, and character n-gram similarity between the normalised POS mention and each candidate. This approach provides a plug-and-play reranker that is orders of magnitude faster than joint-attention cross-encoders and does not require GPU resources, making it appropriate for an undergraduate project context. The Consensus Judger is applied selectively to high-ambiguity inputs only.

2.3 Table-Structured Entity Linking: RoCEL

2.3.1 Overview of the Paper

Wang et al. (2024) [3] introduce RoCEL (Row-Column Entity Linking), which reframes entity linking for tabular data. The row context provides descriptive properties of a target entity; the column context captures relational signals from co-occurring entities. RoCEL encodes both axes via a structured prompt template, yielding significant accuracy improvements over flat-text baselines.

2.3.2 Relevance to Retail Basket Context

A customer receipt maps naturally onto RoCEL's row-column paradigm: each row is a purchased item with its properties, and the basket collectively defines the column context. The presence of "Baby Formula", "Nappies", and "Wipes" in the same transaction provides strong evidence that "Pdr Scent-F" refers to baby powder rather than cosmetic face powder.

2.3.3 Adaptation for This Project

For each ambiguous POS item, the system constructs a structured prompt presenting the item's own properties (row context) alongside a compact comma-separated list of co-purchased items (column context). This context is assembled during Dask preprocessing with negligible overhead and is evaluated against a context-free ablation baseline.

2.4 Distributed Computing Infrastructure

2.4.1 Dask for Parallel Data Processing

Dask [4, 8] is an open-source parallel computing library for Python that constructs a lazy task graph of computational dependencies and schedules them across available CPU cores. For this project, Dask partitions the incoming POS stream into balanced chunks, enabling normalisation, abbreviation expansion, and context construction to proceed in parallel with minimal data movement.

2.4.2 Ray Core for Distributed Orchestration

Ray Core [5] provides three primitives sufficient for the pipeline's orchestration needs: Tasks (stateless remote functions dispatched to worker processes), actors (stateful Python classes with persistent in-memory state that is used to hold the live product catalogue and embedding index), and objects (immutable values stored in Ray's shared-memory object store and accessible across workers without serialisation overhead). A lightweight FastAPI server exposes the Actor pool as a REST endpoint for downstream consumers, avoiding the operational complexity of container orchestration tools such as KubeRay.

2.4.3 Retrieval Libraries: BM25, FAISS, and LightGBM

Lexical retrieval is implemented using BM25 via the rank_bm25 Python library, which indexes the normalised product catalogue at startup and supports sub-millisecond query times. Semantic retrieval uses a bi-encoder [6] with FAISS [7] for approximate nearest-neighbour search over pre-computed catalogue embeddings. The LightGBM reranker [9] is trained on labelled (mention, candidate) pairs using Levenshtein distance, Jaccard similarity, and cosine similarity as features and is served directly from a Ray actor without GPU requirements.

2.5 Summary Comparison of Baseline Works

Table 2.1 provides a structured comparison of the three primary baseline papers.

Criterion	LLM4BioEL [1]	OneNet [2]	RoCEL [3]
Domain	Biomedical	General / Product	Tabular / Web
Fine-Tuning Required	No	No	No
Hallucination Control	Yes (try decoding)	No	No
Retrieval Method	Entropy-based	Bi-encoder + cross-encoder	Not addressed
Context Modelling	Single mention	Single mention	Row-column
Distributed Exec.	Not addressed	Not addressed	Not addressed
Adaptation Here	RCU reference-swap via Ray Actor + Outlines FSM decoding	Hybrid BM25 + bi-encoder (RRF [13]) with LightGBM reranking	Basket-level row-column context encoding

Table 2.1: Comparative summary of primary baseline works.

Chapter 3: Proposed Methodology

This chapter presents the architecture of all five pipeline stages, preceded by a formal problem definition. Each stage is described with explicit attention to both the NLP algorithms and the PDC mechanisms employed.

3.1 Formal Problem Definition

Let M denote the set of all entity mentions, where each mention $m \in M$ is a noisy free-text POS description entered by a checkout operator (e.g., "Ck 12 oz" or "Brd Wh Slcd"). Let $C = \{c_1, c_2, \ldots, c_{|C|}\}$ denote the product catalogue, where each entry c_i consists of a unique SKU identifier s_i, a normalized product name n_i, and a set of descriptive attributes A_i.

The entity linking task is to learn a mapping $f : M \times B \rightarrow C$, where B denotes the basket context — the set of co-occurring mentions in the same transaction — such that $f(m, B) = c^*$ correctly identifies the product described by m. The hard constraint $f(m, B) \in C$ for all inputs is enforced structurally via Outlines-guided FSM decoding [11]. Quality is measured using Accuracy@1, defined as the fraction of test mentions for which f(m, B) returns the ground-truth catalogue entry as the top-ranked prediction. End-to-end latency is defined as the elapsed time between the arrival of a POS event and the production of a fully linked output record.

3.2 System Architecture Overview

The pipeline ingests a continuous stream of POS transactions and produces, for each line item, a verified SKU along with a confidence score and normalised product name. The five stages are (1) Distributed Ingestion and Stream Partitioning, (2) Parallel Text Normalisation and Context Construction, (3) Hybrid Retrieval and Feature-Based Reranking, (4) Outlines-Guided Constrained Generation, and (5) Orchestration with Ray Core and FastAPI. Stages 1–2 are CPU-bound and use Dask [4, 8]. Stages 3–4 are managed by Ray Core Actors [5]. Stage 5 binds the pipeline together and exposes a REST API.

3.3 Stage 1: Distributed Ingestion and Stream Partitioning

3.3.1 Input Format and Ingestion

Raw POS data arrives as a continuous event stream from store terminals. Each event is a JSON record containing a transaction identifier, a timestamp, a terminal identifier, and an array of line items. Each line item contains a free-text product description (the entity mentioned to be linked), a quantity, and a unit price. An Apache Kafka

message queue buffers incoming events, decoupling stream producers from the processing pipeline and providing a persistent replay log.

3.3.2 Dask Stream Partitioning

Events are consumed in micro-batches (typically 500–2000 transactions) and converted into Dask DataFrames [4], partitioned into balanced chunks, one per available CPU core. A hash of the transaction identifier serves as the partitioning key, co-locating all line items of the same transaction on the same worker to enable basket context construction without cross-worker communication in Stage 2.

3.4 Stage 2: Parallel Text Normalization and Context Construction

3.4.1 Text Normalization

Within each Dask partition [4, 8], a normalisation function is applied to every line item in parallel. The function performs case standardisation (lowercase), punctuation normalisation, abbreviation expansion via a retailer-specific lexicon (e.g., "Ck" → "Coke", "Wh" → "Whole", "12oz" → "12 ounce"), numeric unit standardisation, and spell correction constrained to the product vocabulary. Because POS strings often lack whitespace entirely and contain mangled punctuation, the normalisation pipeline employs a character-level tokenisation strategy that identifies token boundaries directly from the raw character stream rather than relying on space-delimited word splitting, making it robust to the concatenated shorthand typical of operator-entered descriptions.

Normalisation throughput is expected to scale linearly with available CPU cores. Based on preliminary profiling, a target throughput of approximately 5,000 line items per micro-batch in under one second is projected for a 16-core cluster processing batches of 1,000 transactions averaging 5 line items each; this figure will be validated empirically in the final evaluation.

3.4.2 Basket Context Construction (RoCEL Adaptation)

Following normalisation, the system constructs basket-level context for each line item, adapting the row-column encoding from RoCEL [3]. For each target mention m, a context object is built with two components. The row context comprises the item's normalised description, quantity, price, and department code. The column context comprises the normalised descriptions of all other basket items B, formatted as a compact comma-separated list. This context object is serialised and passed to Stage 3 with negligible overhead, as transaction items are co-located on the same Dask worker.

3.5 Stage 3: Hybrid Retrieval and Feature-Based Reranking

3.5.1 Hybrid BM25 and Bi-Encoder Retrieval

Upon completion of normalisation, each line item is represented by a normalised text string and a basket context object. The retrieval stage narrows the search space from the full catalogue C to a small candidate shortlist using two complementary systems running in parallel within a Ray Actor.

The first system is a BM25 index (via the rank_bm25 library) built over the product catalogue. BM25 provides high precision on exact lexical tokens such as brand codes, unit sizes (e.g., "12 oz" and "1 L"), and product category codes, which are predictable and formulaic in retail POS data. The second system is a bi-encoder [6] that maps the normalised description to a dense vector and retrieves the top-k semantically similar candidates from a FAISS [7] index of pre-computed catalogue embeddings, providing semantic flexibility for abbreviations and paraphrases not captured by exact matching.

The top candidates from both systems are merged using Reciprocal Rank Fusion (RRF) [13] or, alternatively, a weighted linear combination with a tunable mixing parameter α that balances keyword precision against semantic recall. The top-20 merged candidates are retained for reranking.

3.5.2 LightGBM Feature-Based Reranking

The top-20 candidates from the hybrid retrieval step are passed to a LightGBM classifier that computes a relevance score for each (mention, candidate) pair. The feature vector for each pair includes: Levenshtein edit distance between the normalised mention and the candidate name, Jaccard token overlap, character n-gram similarity, (n=3), cosine similarity of the bi-encoder embeddings, BM25 score, and the candidate's prior frequency in the training corpus. The classifier is trained on labelled pairs from a held-out training split and served from a Ray actor without GPU requirements.

Because the two retrieval systems operate independently, a candidate may appear in only one of the two result lists. To handle this asymmetry, missing features are imputed with a default score of 0.0: a candidate retrieved by BM25 but absent from the bi-encoder results receives a cosine similarity of 0.0 and a vector-space rank of 0, while a candidate retrieved only by the bi-encoder receives a BM25 score of 0.0 and a BM25 rank of 0. This zero-imputation strategy is consistent with the interpretation that an absent score indicates no evidence of relevance from that modality, and it allows the LightGBM classifier to learn modality-specific reliability weights from the training data.

LightGBM reranking is orders of magnitude faster than joint-attention cross-encoders and requires no specialised hardware, making it suitable for an undergraduate implementation context. The top-3 candidates from this stage are retained as the shortlist for Stage 4. The

basket context B is incorporated as additional features in the LightGBM classifier, capturing co-purchase signals at the reranking stage.

3.6 Stage 4: Outlines-Guided Constrained Generation

3.6.1 Inference Routing

Before invoking the full LLM, an inference router inspects the LightGBM score distribution for the top-3 candidates. If the top candidate's score exceeds a confidence threshold and the gap to the second candidate is sufficiently large, the item is returned directly without LLM invocation. Empirical estimates suggest 50–60% of inputs can be resolved at this stage, yielding the target 40% reduction in per-transaction computational cost.

3.6.2 Outlines FSM-Constrained Decoding

For inputs not resolved by the router, the system invokes the LLM with constrained generation managed by the Outlines library [11]. Rather than implementing a custom logit processor, Outlines accepts the list of valid candidate names from the stage 3 shortlist and automatically constructs a finite-state machine (FSM) representing all valid token sequences that could produce one of the candidate strings.

At each token generation step t, Outlines intercepts the LLM's raw logit vector and applies a bitmask derived from the current FSM state: tokens corresponding to valid FSM transitions retain their original logit values, while all other tokens are assigned a logit of negative infinity before the softmax is applied. This bitmask operation assigns zero probability to any token sequence that does not correspond to a valid catalogue entry, providing a mathematically guaranteed correct-format rate of nearly 100% without requiring any custom C++ or low-level Python logit processor code. Because the FSM is constructed from only the top-3 shortlisted candidates rather than the full catalogue, construction overhead is negligible per request.

To handle the edge case of newly stocked items that have not yet synchronised with the local catalogue, an <UNKNOWN_SKU> path is dynamically injected into the FSM valid choices. If the Hybrid Search scores fall below a predetermined confidence threshold, the prompt instructs the LLM to output <UNKNOWN_SKU>. This routes the item to a human-review queue, preventing the system from being forced to hallucinate an incorrect match for out-of-vocabulary products.

3.7 Stage 5: Orchestration with Ray Core and FastAPI

3.7.1 Ray Core Actor Architecture

The complete pipeline is orchestrated using Ray Core primitives rather than the heavier Ray Serve or KubeRay deployment stack. Three actor classes form the backbone of the system: a CatalogueActor that holds the live BM25 index, FAISS index, and LightGBM model in shared memory; a NormalizationActor that manages the Dask client and dispatches preprocessing tasks; and an InferenceActor that maintains the LLM and Outlines FSM generator. Each Actor is instantiated once per physical core or GPU device and registered in a named Actor pool for round-robin request dispatch.

3.7.2 Catalogue Updates via RCU Reference Swapping

When a catalogue change event is received (new SKUs added or products discontinued), the CatalogueActor constructs a fresh immutable catalogue object including rebuilt BM25 and FAISS indices in the background. Once construction is complete, Python's GIL guarantees that the atomic reference swap within the Actor's internal state variable is invisible to concurrent readers: in-flight requests continue reading the old indices until they complete, at which point Ray's distributed reference counting automatically reclaims the old objects. This Read-Copy-Update strategy provides zero-downtime catalogue updates without locks, barriers, or container orchestration.

3.7.3 FastAPI REST Interface

A lightweight FastAPI server wraps the Ray actor pool and exposes a REST endpoint for downstream consumers. Each incoming POST request carries a JSON-encoded transaction and is dispatched as a Ray remote task to an available actor. The FastAPI server is stateless and can be horizontally scaled behind a standard load balancer. Prometheus metrics, including per-stage latency, queue depth, and error rate, are exported via a dedicated /metrics endpoint for real-time operational monitoring.

3.8 Evaluation Plan

The framework will be evaluated along four dimensions corresponding to the research objectives.

Throughput will be measured as transactions processed per second under simulated load using a replay of the POS-Note Corpus, comparing the parallel Ray + Dask framework against a serial Python baseline.

Validity will be verified by checking every output SKU against the product catalogue, with 100% as the target, directly validating the Outlines FSM hallucination guarantee.

Linking accuracy will be measured using Accuracy@1. The primary evaluation benchmark is SynEL (Synthetic Entity Linking), a dataset constructed using the nlpaug library [12] to apply controlled corruptions to a clean product catalogue. Two corruption strategies are applied: Phonetic/Orthographic Shorthand replaces full words with operator-style abbreviations (e.g., "Brd" for bread, "Ck" for Coke) using keyboard-distance character substitutions tunable via the aug_char_p parameter and attribute removal randomly dropping volume descriptors, brand names, and packaging information. The NLPaug approach yields fully reproducible, configurable noise, unlike LLM-based generation. The Retail-786k dataset [10] is retained as a secondary benchmark providing real-world POS coverage.

Computational efficiency will be measured as the fraction of inputs resolved without invoking the full LLM, reported alongside the accuracy impact of routing.

Statistical significance of accuracy improvements will be assessed using McNemar's test. Latency distributions (median and 95th percentile) will be reported per stage and end-to-end to confirm compliance with the 500-millisecond target.

References

[1] Lin, Y., et al. (2025). LLM4BioEL: Restrictive Decoding for Biomedical Entity Linking. Proceedings of the Annual Conference of the Association for Computational Linguistics (ACL 2025).

[2] Liu, X., et al. (2024). OneNet: One-Shot Entity Linking Without Fine-Tuning. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024).

[3] Wang, Z., et al. (2024). RoCEL: Row-Column Entity Linking for Tabular Data. Proceedings of the 2024 ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

[4] Rocklin, M. (2015). Dask: Parallel Computation with Blocked Algorithms and Task Scheduling. Proceedings of the 14th Python in Science Conference (SciPy 2015), pp. 130–136.

[5] Moritz, P., et al. (2018). Ray: A Distributed Framework for Emerging AI Applications. Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2018).

[6] Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP 2020, pp. 6769–6781.

[7] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 7(3), pp. 535–547.

[8] Dask Team. (2026). Parallel Processing in Python with Dask. Retrieved from https://dask.org

[9] Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems (NeurIPS 2017), pp. 3146–3154.

[10] Retail-786k Dataset. (2023). Retail Product Description and SKU Matching Benchmark. Retrieved from the dataset repository.

[11] Willard, B. T., & Louf, R. (2023). Efficient Guided Generation for Large Language Models. arXiv preprint arXiv:2307.0970 2. [Outlines library]

[12] Ma, E. (2019). nlpaug: A Python Library for Text Augmentation. Retrieved from https://github.com/makcedward/nlpaug

[13] Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 758–759.